

G11 · NSTM — 기억을 “언제 고치고 언제 읽을지” 를 분리하면 생기는일

이 권은 arXiv:2607.15271 「Online Neural Space Time Memory for Dynamic Novel View Synthesis」 (Elmieh et al., UW/Google — Behrouz·Li 공저) 를 다룬다. 겉보기엔 영상 (novel view synthesis) 논문이지만, 속은 이시리즈의 언어다: TTT fast weight, L2 inner loss, Muon, 그리고 G10 에서 배운 memory caching 을 실전 투입해 성공·실패 조건까지 실험으로 보여 준 논문이다. 후반부 (11.6~11.9) 는 이 논문이 HOPE/Nested Learning(G07)·Sleep(G08)·memory caching(G10) 에 주는 힌트를 정리한다 — 이 부분은 원저자의 주장이 아니라 우리의 해석임을 미리 밝힌다.

핵심 결론부터: 이 논문은 NL/Sleep 의 미해결 문제 (무엇을 어떻게 느린 메모리로 옮길 것인가) 를 직접 풀지는 않는다. 대신 세 가지 강한 힌트를 실험으로 준다 — ① 쓰는 (갱신) 파라미터와 읽는 파라미터를 분리하라, ② 불변 지식과 상황 종속 상태를 구조적으로 분리해야 consolidation (평균) 이 성립한다, ③ 입력을 차단하고 기억만으로 답하게 훈련해야 진짜 parametric memory 가 생긴다.

11.1 무엇을 하는 논문인가

카메라 스트림으로 움직이는 사람을 보며 실시간으로 novel view 를 합성한다. 지금 보이는 부분은 현재 입력으로, 가려진 부분 (돌아선 등) 은 기억으로 복원한다. 요구사항이 이 시리즈의 서빙 문제와 똑같다: 기억은 분 (minute) 스케일로 유지되어야 하고, 처리량은 실시간 (프레임률) 이어야 한다.

기존 TTT 계열을 그대로 쓰면 두 가지가 무너진다 — 매 프레임 gradient 갱신이라 느리고, 긴 지평에서 반복 갱신하면 fast weight 가 drift 로 불안정해진다. NSTM 의 처방 세 가지:

1. **Memorization-Synthesis 분리** — 갱신은 낮은 빈도 (초당 1 회), 읽기는 매 프레임.
2. **Memory Loss ($\mathcal{L}_{mem}$)** — 입력을 차단하고 기억만으로 복원하게 하는 감독.
3. **Memory Caching** — 과거 스냅샷 평균을 읽어 drift 를 억제 (G10 의 기법을 채택).

11.2 구조 — 어떤 파라미터가 온라인으로 변하는가

24 개 Transformer block, 각 block = **cross-view attention** + **TTT memory** + **FFN**. TTT 메모리는 bias 없는 3-행렬 SwiGLU MLP:

$$f_W(x) = (\text{SiLU}(W_1 x) \odot W_3 x) W_2, \quad W = \{W_1, W_2, W_3\}$$

갱신은 L2 inner loss 의 단일 gradient 스텝 $W' = W - \eta \nabla_W \|v - f_W(k)\|_2^2$, 여기에 **Newton-Schulz 직교정규화 (Muon)** 와 **L2 weight normalization** 으로 안정화.

종류	예	추론 중
slow weights Θ	attention·FFN·projection	고정
fast 초기값 W_{init}	W 의 시작점	고정 (outer-loop 에서 학습)
active fast weights	현재의 W_1, W_2, W_3	주기적으로 갱신
cached memory state	과거 스냅샷의 running mean	읽을때 혼합

기호 풀이 이 시리즈의 $W(\text{fast})/\Theta(\text{slow})$ 구분이 그대로다. 새로 등장하는것은 넷째 줄 — “읽기 전용 그림자 상태” 다. 온라인으로변하는 것은 각 layer 에 심긴 전용 memory MLP 뿐이고, backbone 은절대 건드리지 않는다.

주의 논문에는 전체 파라미터 수와 W 의 hidden 차원이 명시돼 있지 않다 — fast memory 가 정확히 몇 개 파라미터인지는 논문만으로 계산할 수 없다.

11.3 읽기/쓰기 빈도 분리 — 숫자로

단일 H100 기준: memorization(갱신+적용) 58.14ms, synthesis(적용만) 27.01ms. 갱신을 초당 1 회로 두고 그 사이~30 개 프레임이 같은 메모리를 읽으면 amortized $\approx 28.1\text{ms/frame}$ — 실시간이 된다.

시스템 모델링 관점 이것은 G09 와 MEMOIR 에서 계산했던 그 트레이드오프의 실물 이다 — SMT decode 의 지배항이 “매 token per-user weight 읽기+갱신” 이었음을 기억하라. NSTM 은 mutable 모델이라고 매 스텝 update kernel 을 돌릴필요가 없다는 실증이다: read-only 스텝끼리는 서로 독립이므로 ① 일반 decode 는 read-only 상태 기준으로 batching, ② 갱신이 필요한 요청은 별도 update queue, ③ 갱신 완료 후 새 state 버전으로 스위치 — RW-TTT(owner/version 태깅) 와 정확히 합치되는 스케줄이다. 단, 요청마다다른 W 를 쓰는 LLM 서빙의 grouped-GEMM/state-paging 문제까지 푼 것은아니다.

11.4 Memory Caching 실전 — active 와 stable-read 의 이중상태

G10 의 기법이 그대로 온다. K 번째 갱신마다 스냅샷을 저장하고 running mean 을 유지:

$$\bar{W}_n = \frac{n-1}{n}\bar{W}_{n-1} + \frac{1}{n}W_{nK}, \quad \hat{W}_{t-1} = \frac{W_{t-1} + m\bar{W}_m}{m+1}$$

쓰기는 active W 에 계속, 읽기는 안정화된 $\hat{W}$ 로, 평균을 active 에 되쓰지는 않는다. 추가 비용은 파라미터 크기 상태 한 세트.

직관 이것은 NL 이 말하는 fast $\rightarrow$ slow 지식 이전이 아니다. “계속 변하는 active fast-weight 와, 역사적으로 안정화된 그림자 (shadow) fast-weight 를동시에 유지” 하는

구조다 — 주식의 현재가와 이동평균선을 같이 보는것과 같다. 현재가 (active) 로 거래 (쓰기) 하되, 추세 판단 (읽기) 은이동평균 (cached) 으로.

11.5 가장 중요한 실험 — 파라미터 평균은 아무 때나 작동하지않는다

ablation(PSNR, Memory Stress Test):

구성	PSNR
LaCT-NVS (dot-product loss)	22.32
LaCT-NVS + L2 loss	27.77
LaCT-NVS + L2 + memory caching	20.25 (붕괴!)
NSTM w/o $\mathcal{L}_{mem}$	28.23
NSTM w/o caching	29.24
Full NSTM	30.09

같은 **memory caching** 이 LaCT 에서는 재앙 ($27.77 \rightarrow 20.25$), NSTM 에서는이득 ($29.24 \rightarrow 30.09$) 이다. 왜인가? LaCT 의 fast weight 에는 물체 identity/외형과 현재 pose·deformation·시점 종속 상태가 뒤섞여있다. 이걸 시간 평균하면 여러 포즈가 파라미터 공간에서 겹쳐 “평균포즈” 에 갇힌다. NSTM 은 역할을 분리했다 — **fast memory** 에는 비교적불변인 identity·appearance, 현재 pose·deformation 과의 정렬은 **cross-view attention**. 그래서 스냅샷 평균이 의미를 유지한다.

핵심 파라미터 consolidation(평균·souping·merge) 은 “같은 종류의정보” 를 담은 파라미터끼리만 성립한다. consolidation 의 성패는 update rule 이 아니라, 그 전에 기억을 invariant + context-dependent 로 분해해두었는가에 달려 있다 — 이 권에서 가져갈 단 하나의 문장이다.

11.6 Memory-only supervision — “진짜 기억했는지” 검사하는법

학습 중 짝수 timestep 에서는 target token 이 현재 입력 token 을 attention 으로 볼 수 없게 차단하고, 오직 fast memory 만으로 이미지를복원하게 한다 ($\mathcal{L}_{mem}$). 홀수 timestep 은 정상 synthesis. $\mathcal{L}_{mem}$ 을 빼면 $30.09 \rightarrow 28.23$ — 우회로를 열어 두면 정보가파라미터에 실제로 들어가지 않는다.

시스템 모델링 관점 (LLM 번역). LLM 의 online memory 도 똑같이게으르다 — KV cache, 원문 context, retrieval, residual stream 이라는우회로가 있으면 fast weight 에 쓰지 않는다. 처방: 학습/평가 중 주기적으로 **KV·context·retrieval** 을 차단하고 **parametric memory** 만으로 이전사실을 복원시켜라. $\mathcal{L}_{memory-only} = \mathcal{L}(f_{\theta,w}(q; \text{context masked}), y)$. 이 loss 가 낮아야 “읽을 수 있었던 것” 이 아니라 “내재화된 것” 이다 — NL/Sleep 에서 모호했던 “무엇을 consolidation 대상으로 인정하나” 에대한 쓸 만한 operational criterion 이다.

11.7 HOPE/NL·Sleep 에 주는 힌트 (해석)

Nested Learning(G07) 에 대해 — NL 은 다주파수 구조를 이론으로 제시했지만 구현 세부 (무엇을 · 언제 · 어떻게 이전하나) 가 열려 있었다. NSTM 은 그중 한 단계의 구체적인 구현 예다:

NL 의 미해결점	NSTM 의 힌트
매 token 갱신해야 하나	읽기/쓰기 빈도 분리 (1Hz 갱신 + 매 프레임 읽기)
fast memory drift 를 어떻게 막나	active state 와 historical read state 분리
무엇을 consolidation 하나	불변 정보만 memory 파라미터에
문맥마다 기억이 달라지는 문제	별도 정렬 모듈 (cross-view attention) 에 위임
진짜 파라미터에 저장됐나	context 차단 memory-only loss
갱신 안정성	L2 objective + Muon + weight normalization

Sleep(G08) 에 대해 — Sleep 은 “어디로 옮길 것인가” (더 느린 블록의 새 low-rank expert) 를 제안했지만, **sleep 직전에 무엇을 teacher 로 삼을 것인가**는 모호했다. 가장 최근의 active fast-weight 를 그대로 teacher 로 삼으면 최근 샘플 편향 · norm drift · 일회성 오염이 섞인다. NSTM 의 힌트: teacher 는 순간의 active 상태가 아니라 **시간적으로 안정화된 stable-read 상태 (스냅샷 앙상블)** 여야 한다. 그러면 자연스러운 3 단 lifecycle 이 나온다:

$$W_{\text{active}} \rightarrow W_{\text{stable-read}} \rightarrow W_{\text{slow expert}}$$

— 즉각 적응 → 여러 시점에 걸쳐 검증된 안정 기억 (pre-consolidation buffer) → sleep 에서 영구 통합. NSTM 의 cached state 는 완성된 consolidation 이 아니라 **sleep 전 대기 버퍼**로 읽는 것이 적절하다.

Memory Caching(G10) 에 대해 — G10 이 남긴 질문 (“스냅샷 평균은 언제 안전한가”) 에 NSTM 의 ablation 이 답을 준다: **분해가 안 된 fast weight(LaCT) 의 평균은 붕괴하고, 분해된 fast weight(NSTM) 의 평균은 강력한 안정화 장치**가 된다.

주의 이 절은 우리의 해석이다. NSTM 은 fast → slow(backbone) 이전을 하지 않으므로 엄밀한 의미의 NL online consolidation 이 아니며, LLM 에의 대응 (adapter·router·context adapter 등) 은 검증되지 않은 유추다.

11.8 두 논문을 합치면 — 가능한 설계 스케치 (추론)

Wake(온라인): 각 layer 에 $\{\Theta, W_{\text{fast}}, W_{\text{stable}}\}$ + context adapter. 읽기는 $W_{\text{read}} = g_t W_{\text{fast}} + (1 - g_t) W_{\text{stable}}$ (novelty/confidence 게이트). 쓰기는 고정 주기 대신 **surprise 임계 · 불일치 · memory-only probe 실패 · episode 경계**에서만 (논문 스스로 고정주기의 “갱신 사이 사건 놓침” 을 한계로 인정 — learned trigger 가 낫다). Pre-sleep: 여러 문맥에

서 반복되고, memory-only probe 를 통과하고, 스냅샷간 예측이 일치하는 정보만 stable 로. 단순 평균 대신 Fisher-가중 merge·functional distillation·subspace projection 도 후보. Sleep(G08 그대로): teacher = $f_{\theta, W_{\text{stable}}}$, student = 새 low-rank expert 만 학습 → 통합 후 fast reset·stable 일부 비우기 · 새 expert 활성화.

11.9 이 논문이 해결하지 못한 것 (정직하게)

1. 진정한 fast → slow consolidation 이 아니다 — 스냅샷 평균을 읽을뿐 backbone·저빈도 메모리를 학습시키지 않는다.
2. 의미 단위 기억 선택이 없다 — 무엇이 중요한 사실인지 semantic 판단은 없다. 영상에서는 아키텍처가 identity/motion 을 어느 정도 자동분리해 준 것.
3. 쉬운 조건이다 — 같은 장면 · 같은 identity. 충돌하는 사실, 시간에 따라 변하는 지식은 실험하지 않았다.
4. 평균의 전제가 강하다 — 모든 스냅샷이 같은 초기화 · 연속궤적 · 동일 parameter permutation 위에 있다. 독립 학습된 메모리들의 평균근거는 아니다.
5. primacy bias — 균일 평균은 초기 기억을 과대 대표하고, 갱신사이 사건을 놓칠 수 있다 (저자 인정).
6. 분 스케일 검증이다 — 60 timestep 수준. 수일 · 수개월 lifelong 증거는 없다. 다만 LaCT 가 장기에서 무너지는 동안 ~30 PSNR 을 유지하는 제한적 장기 안정성 증거는 있다.

핵심 이 논문에서 가져갈 연구 가설 한 줄: “online consolidation 의관건은 update rule 이 아니라, consolidation 전에 기억을 invariant/contextual 성분으로 분해하는 것이다.” 분해가 없으면평균·EMA·merge·distillation 전부 ‘여러 포즈의 평균’처럼 붕괴할 수 있고, 분해가 되면 단순 산술 평균조차 강한 장기 안정화 장치가 된다.